

Preventing Context Loss in Long Conversations

When we continuously send full conversation history to an LLM, eventually the context window becomes full.

When that happens:

- Older messages get removed
- Important information may be lost
- The model may “forget” earlier discussions

So the key question is:

How do we protect important data instead of losing it?

How do we reduce token usage?

Let us understand the solutions clearly.

1. Solution 1: Conversation Summarization (Context Condensation)

Instead of sending every single past message individually, we can summarize older conversations.

Example:

Suppose we have 20 old conversation messages that are about to be removed due to context window limits.

Instead of:

Sending all 20 messages again,

We:

1. Generate a summary of those 20 messages.
2. Replace them with a single summarized version.
3. Send the summary along with the current prompt.

Result:

- Fewer tokens used
- Important information preserved
- Context window saved
- Reduced cost

This technique is called Context Condensation.

It compresses conversation history into a smaller representation while keeping key meaning.

2. Prompt Engineering vs Context Engineering

In traditional prompt engineering:

- All previous messages are passed directly to the LLM.
- It does not check if the messages are relevant.
- It simply sends everything.
- Token usage increases quickly.

If you suddenly change topic:

All previous unrelated content is still sent to the LLM.

This wastes tokens and may confuse the model.

Context Engineering

Context engineering is more structured and intelligent.

Instead of sending everything:

- We store conversations properly.

- We retrieve only relevant information.
- We send only what is needed for the current question.

This improves:

- Accuracy
- Token efficiency
- Cost
- Response quality

3. How Context Engineering Works

In context engineering, we do not rely only on conversation buffer.

Instead, we:

1. Store conversation data or documents in a database.
2. Convert them into embeddings.
3. Store them in a Vector Database.
4. When a new question is asked:
 - Retrieve only relevant chunks.
 - Combine them with the current prompt.
 - Send that to the LLM.

This ensures:

Right question + Right data = Better answer

This technique is commonly called:

RAG (Retrieval Augmented Generation)

4. Prompt Engineering vs Context Engineering (Comparison)

What is it?

- Prompt Engineering: Crafting better instructions.
- Context Engineering: Supplying the right external data.

Main Focus:

- Prompt Engineering: How to ask.
- Context Engineering: What information to give.

Knowledge Source:

- Prompt Engineering: Model's internal training.
- Context Engineering: Your documents, database, or knowledge base.

Best Use Case:

- Prompt Engineering: Tone, formatting, style, logic.
- Context Engineering: Accuracy and company-specific answers.

Hallucination Prevention:

- Prompt Engineering: Medium.
- Context Engineering: Very High.

Tools Used:

- Prompt Engineering: Templates, instructions.
- Context Engineering: RAG, vector stores, embeddings.

Who Needs It?

- Prompt Engineering: Everyone.
- Context Engineering: Teams building production-grade AI systems.

5. Simple Explanation

Prompt Engineering:

Passing everything to the LLM and improving how we ask.

Context Engineering:

Passing only what is required to the LLM.

6. Example to Understand Clearly

Only Prompt Engineering:

You ask:

“Explain my company’s microservices deployment architecture with EKS + ArgoCD.”

If you do not provide company documents:

The model guesses.

It may hallucinate.

With Context Engineering:

You first provide:

- Your EKS architecture document
- Your ArgoCD configuration
- Your GitOps workflow

Then ask:

“Explain my architecture in simple words.”

Now the model:

- Reads your real documents
- Uses your data
- Produces accurate output

This greatly reduces hallucination.

7. Reducing Token Usage Strategies

Here are practical ways to reduce tokens:

1. Summarize older conversations.
2. Store conversations externally (database or vector DB).
3. Retrieve only relevant content.
4. Avoid sending unrelated history.
5. Chunk large documents.
6. Use embeddings for semantic search.
7. Remove redundant system prompts.

8. Orchestration Stack

As AI systems evolved, simple prompt-based systems were not enough.

Modern AI systems use orchestration frameworks that include:

- RAG pipelines
- Semantic memory stores
- Agent frameworks
- Feedback loops
- Controllers and executors
- Conditional logic
- Tool calling mechanisms

This is called Orchestrated Intelligence.

Instead of focusing only on the model, we design systems around it.

The industry is shifting from:
Model-Centric AI
to
System-Orchestrated Intelligence

9. Final Summary

Problem:

Conversation buffer gets exhausted → data loss.

Solution:

- Summarize old conversations.
- Store knowledge in vector databases.
- Retrieve only relevant context.
- Reduce token usage.
- Use context engineering instead of blindly sending everything.

Key Idea:

Prompt Engineering improves how we ask.

Context Engineering improves what we send.

In real-world AI systems, context engineering is critical for:

- Scalability
- Accuracy
- Cost control
- Enterprise-level reliability